

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства
Intel – обработка символьных данных»

Выполнил:

Студент 4 курса

Группы АС-66

Рогожин А. С.

Проверила:

Кулик А. Д.

Цель работы: изучение программной модели микропроцессора, практическая работа с подсистемой регистровой и оперативной памяти.

Вариант 20

Постановка задачи. Дан текст – непустая последовательность не длиннее ста символов. Признаком конца ввода является точка, в сам текст точка не входит.

Проверить, удовлетворяет ли текст заданному условию. Если условие выполнено, преобразовать текст по указанному в варианте правилу; в противном случае выдать соответствующее сообщение. Преобразованный текст напечатать.

Проверяемое условие и правила обработки текста определяются конкретным вариантом задания.

Если введенная последовательность символов не является текстом, преобразовывать ее не нужно, а следует напечатать соответствующее сообщение.

Ввод текста, проверка условия, обработка текста и печать результата должны выполняться последовательно, отдельными частями программы.

Используя знания и навыки, полученные в результате выполнения лабораторной работы №1, а также изучив дополнительные теоретические материалы, приведенные в методическом указании, написать программу на ассемблере, реализующую обработку текста согласно варианту, выданному преподавателем.

Проверяемое условие: текст содержит не менее трех латинских букв.

Правило преобразования: заменить каждую прописную латинскую букву следующей за ней по алфавиту, букву Z менять на A.

Требования к программе: нельзя использовать дополнительную память, размер которой зависит от длины текста (в частности, стек) во всех вариантах, кроме тех, где явно не указано обратное.

При реализации вариантов, помеченных "с.к.", требуется использовать строковые команды с префиксами повторения.

Проверку условия и выполнение преобразований текста следует реализовать в виде процедур. Глобальные переменные не использовать.

Интерфейс программы должен быть достаточно удобным: надо печатать приглашение для ввода, пояснять, какое условие проверяется, истинно ли оно и как текст преобразуется.

Рекомендации: в некоторых вариантах удобно реализовать в качестве процедур или макросов проверку, является ли символ буквой, цифрой и т.п.

В процедуре преобразования текста может быть полезным иметь два указателя на позиции в тексте: откуда происходит чтение и куда идёт запись.

Максимально возможную длину текста описать в виде константы.

Код программы

```
.386
.model flat,stdcall
option casemap:none

includelib kernel32.lib

ExitProcess PROTO :DWORD
GetStdHandle PROTO :DWORD
ReadConsoleA PROTO :DWORD,:DWORD,:DWORD,:DWORD,:DWORD
WriteConsoleA PROTO :DWORD,:DWORD,:DWORD,:DWORD,:DWORD
SetConsoleCP PROTO :DWORD
SetConsoleOutputCP PROTO :DWORD

STD_INPUT_HANDLE EQU -10
STD_OUTPUT_HANDLE EQU -11
MAX_LEN EQU 100

.const

prompt db "Vvedite tekst (do 100 simvolov, konec - '.'): ",0
nottext db 13,10,"Vvedennaya posledovatelnost ne yavlyaetsya tekstom.",13,10,0
condition db 13,10,"Proveryaem uslovie: tekst sodержit ne menee treh latinskih bukv.",13,10,0
true_msg db "Uslovie vpolneno. Preobrazuem tekst:",13,10,0
false_msg db "Uslovie ne vpolneno (menee treh latinskih bukv).",13,10,0
original db "Ishodny tekst: ",0
result db 13,10,"Preobrazovannyi tekst: ",0
newline db 13,10,0

.code

PrintString PROC
    push eax
    push ecx
    push edx
    push esi
    push edi

    mov esi,edx
    xor ecx,ecx

PRINT_LOOP:
```

```
    cmp byte ptr [esi],0
    je PRINT_DONE
    inc esi
    inc ecx
    jmp PRINT_LOOP
```

PRINT_DONE:

```
    push STDOUT_HANDLE
    call GetStdHandle
    mov edi,eax
```

```
    sub esp,4
    mov eax,esp
```

```
    push 0
    push eax
    push ecx
    push edx
    push edi
    call WriteConsoleA
```

```
    add esp,4
```

```
    pop edi
    pop esi
    pop edx
    pop ecx
    pop eax
    ret
```

PrintString ENDP

ReadText PROC

```
    push ebp
    mov ebp,esp
    sub esp,16
```

```
    push ebx
    push esi
    push edi
```

```
    mov esi,edx
    xor ebx,ebx
    mov dword ptr [ebp-16],0
```

```
push STD_INPUT_HANDLE
call GetStdHandle
mov edi,eax
```

READ_LOOP:

```
lea edx,[ebp-4]
```

```
push 0
```

```
lea eax,[ebp-8]
```

```
push eax
```

```
push 1
```

```
push edx
```

```
push edi
```

```
call ReadConsoleA
```

```
test eax,eax
```

```
jz READ_ERROR
```

```
movzx eax,byte ptr [ebp-4]
```

```
cmp eax, '.'
```

```
je READ_END
```

```
cmp eax,13
```

```
je READ_ERROR
```

```
cmp eax,10
```

```
je READ_ERROR
```

```
cmp ebx,MAX_LEN
```

```
jae READ_OVERFLOW
```

```
mov byte ptr [esi],al
```

```
inc esi
```

```
inc ebx
```

```
jmp READ_LOOP
```

READ_OVERFLOW:

```
mov dword ptr [ebp-16],1
```

```
jmp READ_LOOP
```

READ_END:

```
    cmp dword ptr [ebp-16],1
    je READ_ERROR
```

```
    mov byte ptr [esi],0
    mov eax,ebx
    jmp READ_EXIT
```

READ_ERROR:

```
    mov eax,-1
```

READ_EXIT:

```
    pop edi
    pop esi
    pop ebx

    mov esp,ebp
    pop ebp
    ret
```

ReadText ENDP

IsText PROC

```
    push ecx
    push edx
    push esi
```

```
    cmp eax,0
    je TEXT_ERROR
```

```
    mov ecx,eax
    mov esi,edx
```

TEXT_LOOP:

```
    movzx edx,byte ptr [esi]
```

```
    cmp edx,20h
    jb TEXT_ERROR
```

```
    cmp edx,7Fh
    je TEXT_ERROR
```

```
    inc esi
    dec ecx
```

```
jnz TEXT_LOOP
```

```
mov eax,1
```

```
jmp TEXT_EXIT
```

```
TEXT_ERROR:
```

```
xor eax,eax
```

```
TEXT_EXIT:
```

```
pop esi
```

```
pop edx
```

```
pop ecx
```

```
ret
```

```
IsText ENDP
```

```
; Проверка условия: подсчет всех латинских букв (и заглавных A-Z, и строчных a-z)
```

```
CheckCondition PROC
```

```
push ecx
```

```
push esi
```

```
mov ecx, eax      ; ecx = длина текста
```

```
mov esi, edx      ; esi = указатель на текст
```

```
xor eax, eax      ; eax = счетчик латинских букв
```

```
CHECK_LOOP:
```

```
mov dl, [esi]
```

```
; Проверка на заглавные (A-Z)
```

```
cmp dl, 'A'
```

```
jb CHECK_LOWER
```

```
cmp dl, 'Z'
```

```
jbe LETTER_FOUND
```

```
CHECK_LOWER:
```

```
; Проверка на строчные (a-z)
```

```
cmp dl, 'a'
```

```
jb CHECK_NEXT
```

```
cmp dl, 'z'
```

```
ja CHECK_NEXT
```

```
LETTER_FOUND:
```

```
inc eax           ; Нашли латинскую букву
```

CHECK_NEXT:

```
inc esi
dec ecx
jnz CHECK_LOOP
```

```
pop esi
pop ecx
ret
```

CheckCondition ENDP

; Правило преобразования: Каждую прописную латинскую букву заменить следующей, Z заменить на A

TransformText PROC

```
push ecx
push esi
```

```
mov ecx, eax      ; ecx = длина текста
mov esi, edx      ; esi = указатель на текст
```

TRANSFORM_LOOP:

```
mov al, [esi]

cmp al, 'A'
jb TRANSFORM_NEXT
cmp al, 'Y'
ja CHECK_Z        ; Если буква от 'A' до 'Y'

inc al            ; Сдвигаем на следующую букву
mov [esi], al
jmp TRANSFORM_NEXT
```

CHECK_Z:

```
cmp al, 'Z'
jne TRANSFORM_NEXT
mov byte ptr [esi], 'A' ; Если 'Z', заменяем на 'A'
```

TRANSFORM_NEXT:

```
inc esi
dec ecx
jnz TRANSFORM_LOOP
```

```
pop esi
pop ecx
ret
```


TransformText ENDP

start PROC

push ebp

mov ebp,esp

sub esp,116

push 1251

call SetConsoleCP

push 1251

call SetConsoleOutputCP

mov edx,OFFSET prompt

call PrintString

lea edx,[ebp-104]

call ReadText

cmp eax,-1

je ERROR_TEXT

mov [ebp-108],eax ; Сохраняем длину текста

lea edx,[ebp-104]

mov eax,[ebp-108]

call IsText

cmp eax,1

jne ERROR_TEXT

mov edx,OFFSET condition

call PrintString

lea edx,[ebp-104]

mov eax,[ebp-108]

call CheckCondition

; В eax теперь общее количество латинских букв

cmp eax, 3

jnl CONDITION_FALSE ; Если меньше 3, то условие не выполнено

mov edx,OFFSET true_msg

```
call PrintString
```

```
mov edx,OFFSET original
```

```
call PrintString
```

```
lea edx,[ebp-104]
```

```
call PrintString
```

```
; Преобразуем текст (замена прописных по условию)
```

```
lea edx,[ebp-104]
```

```
mov eax,[ebp-108]
```

```
call TransformText
```

```
mov edx,OFFSET result
```

```
call PrintString
```

```
lea edx,[ebp-104]
```

```
call PrintString
```

```
mov edx,OFFSET newline
```

```
call PrintString
```

```
push 0
```

```
call ExitProcess
```

```
CONDITION_FALSE:
```

```
mov edx,OFFSET false_msg
```

```
call PrintString
```

```
push 0
```

```
call ExitProcess
```

```
ERROR_TEXT:
```

```
mov edx,OFFSET nottext
```

```
call PrintString
```

```
push 0
```

```
call ExitProcess
```

```
start ENDP
```

```
END start
```

Результат работы программы:

```
PS D:\Uni\АПО ЭВМиС\АРОЕVMiS_AS66\reports\Rogozhin\lab2\src> D:\Uni\jwas

GoLink.Exe Version 0.26.14 - Copyright Jeremy Gordon 2002/9 - JG@JGnet.c
Output file: lab2.exe
Format: win32 size: 2,048 bytes
PS D:\Uni\АПО ЭВМиС\АРОЕVMiS_AS66\reports\Rogozhin\lab2\src> .\lab2.exe
Vvedite tekst (do 100 simvolov, konec - '.'): Hello, Charlotte.

Proveryaem uslovie: tekst sodержit ne menee treh latinskih bukv.
Uslovie vpolneno. Preobrazuem tekst:
Ishodny tekst: Hello, Charlotte
Preobrazovannyi tekst: Iello, Dharlotte
PS D:\Uni\АПО ЭВМиС\АРОЕVMiS_AS66\reports\Rogozhin\lab2\src> █
```

Вывод: в ходе лабораторной работы была изучена программная модель микропроцессора и получены практические навыки работы с регистрами процессора и оперативной памятью при разработке программы на языке ассемблера. В соответствии с заданным вариантом была разработана программа, выполняющая последовательный ввод текста, проверку корректности введенной последовательности, проверку условия равенства количества прописных латинских и прописных русских букв, а также преобразование текста при выполнении условия.

